

Efisiensi Eksplorasi pada Permainan Minesweeper menggunakan Metode Critic as Explorer (CAE) dalam Kerangka Deep Q-Network (DQN)

Yoel Dwi Miryano¹ and Muhammad Firdaus Syawaludin Lubis, S.T., M.T., Ph.D.²

Dept. of Electrical Engineering

Universitas Indonesia

Depok, Indonesia

yoel.dwi@ui.ac.id, chemfirus@gmail.com

Abstract—Meskipun *Deep Q-Network* (DQN) efektif dalam banyak tugas pengambilan keputusan, strategi eksplorasi ϵ -greedy sering gagal pada *environment* POMDP dengan penalti fatal seperti *Minesweeper* akibat rentannya terminasi dini yang menghambat akumulasi pengalaman. Untuk mengatasi inefisiensi ini, penelitian ini mengintegrasikan metode *Critic as Explorer* (CAE)—strategi eksplorasi intrinsik berbasis ketidakpastian yang memanfaatkan representasi laten jaringan—ke dalam arsitektur DQN. Eksperimen dilakukan pada papan *Minesweeper* 6×6 menggunakan agen berbasis *Convolutional Neural Network* (CNN) yang dilatih selama 80,000 episode. Hasil menunjukkan bahwa CAE-DQN secara signifikan mengungguli *baseline* ϵ -greedy, mencapai rata-rata *win rate* 72.6% berbanding 58.0% ($p < 0.001$, $d = 12.57$). Metode yang diusulkan juga mempercepat efisiensi sampel secara drastis (mencapai *win rate* 50% dalam 6,000 berbanding 65,000 episode), menghasilkan langkah yang lebih terarah dengan peningkatan *progress per click* sebesar 13%, serta menunjukkan perilaku *self-annealing* adaptif. Walaupun sempat teramati fenomena *policy collapse* pada akhir pelatihan akibat dinamika pengusiran data pada *replay buffer*, model pada titik puncak tetap mempertahankan kemampuan generalisasi yang kuat. Temuan ini mengukuhkan efektivitas eksplorasi CAE pada *environment* dengan *sparse reward* dan penalti terminal.

Index Terms—Reinforcement Learning, Deep Q-Network, Exploration, Critic as Explorer, Minesweeper, Sample Efficiency.

I. PENDAHULUAN

Reinforcement Learning (RL), khususnya melalui arsitektur *Deep Q-Network* (DQN), telah menunjukkan keberhasilan yang signifikan dalam menyelesaikan berbagai permasalahan pengambilan keputusan kompleks [1], [2]. Meskipun demikian, implementasi RL pada *environment* yang memiliki karakteristik observasi parsial (*Partially Observable Markov Decision Process* atau POMDP) [3], [4], *sparse reward*, dan penalti terminal yang fatal masih menjadi tantangan besar.

Permainan *Minesweeper* merupakan representasi klasik dari masalah ini. *Minesweeper* telah dibuktikan sebagai permasalahan *NP-complete* [5], di mana *agent* harus menyimpulkan lokasi ranjau berdasarkan informasi angka yang sangat terbatas. Dalam kerangka DQN standar, *agent* umumnya menggunakan strategi eksplorasi ϵ -greedy. Pada *environment* dengan penalti fatal seperti *Minesweeper*, eksplorasi acak ini

sering kali memicu klik pada ranjau di awal permainan, menyebabkan terminasi dini [6]. Akibatnya, *agent* gagal mengumpulkan transisi pengalaman (*experience transitions*) yang panjang dan informatif, sehingga proses pembelajaran tidak efisien secara sampel (*sample-inefficient*).

Untuk mengatasi inefisiensi ini, diperlukan mekanisme eksplorasi yang tidak hanya bergantung pada probabilitas acak, melainkan pada ketidakpastian (*uncertainty*) dari status *agent* tersebut. Metode *Critic as Explorer* (CAE) [7] menawarkan pendekatan eksplorasi yang efektif dan novel dengan memanfaatkan representasi fitur laten dari jaringan *critic*. CAE membangun *Upper Confidence Bound* (UCB) di ruang laten untuk memberikan bonus eksplorasi intrinsik secara aman pada transisi yang memiliki tingkat ketidakpastian epistemik yang tinggi, tanpa memerlukan model yang berat pada *resource* komputasi.

Penelitian ini mengusulkan integrasi metode CAE ke dalam kerangka DQN (CAE-DQN) untuk permainan *Minesweeper*. Kontribusi utama dari makalah ini adalah:

- 1) Mengimplementasikan arsitektur CAE-DQN dengan representasi *one-hot state* spasial pada papan *Minesweeper* berukuran 6×6 .
- 2) Membuktikan bahwa penggunaan bonus eksplorasi CAE meningkatkan *win rate* secara signifikan dan mempercepat efisiensi sampel dibandingkan DQN konvensional.
- 3) Menganalisis dinamika pelatihan *agent*, termasuk mekanisme *self-annealing* pada bonus eksplorasi dan fenomena *policy collapse* yang dipicu oleh pengusiran/penghapusan data (eviction) pada *replay buffer*.

II. TINJAUAN PUSTAKA

A. Penerapan RL pada Minesweeper dan Arsitektur DQN

Dalam perspektif *Reinforcement Learning* (RL), *Minesweeper* menghadirkan tantangan komputasi yang kompleks. Permainan ini diklasifikasikan sebagai *NP-complete* [5] dan diformalkan sebagai *Partially Observable Markov Decision Process* (POMDP) [3], di mana *agent* harus mengambil keputusan dengan informasi parsial dan berisiko penalti terminal.

Studi awal oleh Gardea et al. (2015) mengeksplorasi *tabular Q-learning* pada *Minesweeper*. Meskipun berhasil pada papan 4×4 , performa menurun drastis di bawah 5% [8] pada konfigurasi lebih besar akibat ledakan dimensi *state space*. Keterbatasan ini memotivasi transisi ke *Deep RL*. *Deep Q-Network* (DQN) [1] mengaproksimasi fungsi *action-value* optimal $Q^*(s, a)$ menggunakan jaringan saraf berparameter θ . Stabilitas DQN dicapai melalui *experience replay* untuk memutus korelasi temporal, dan *target network* berparameter θ^- untuk mengurangi osilasi saat meminimalkan *Mean Squared Error* (MSE).

Penelitian terbaru oleh Wang & Lei (2025) mengimplementasikan DQN berbasis *Convolutional Neural Network* (CNN) pada *Minesweeper*, dan mencapai *win rate* 93.3% pada papan 6×6 . Namun, konvergensi tersebut baru tercapai setelah sekitar 1.5 juta *training steps*. Tingginya interaksi yang dibutuhkan mengindikasikan bahwa strategi ϵ -greedy konvensional masih kurang efisien secara sampel, sebuah batasan yang diakui secara eksplisit oleh Wang & Lei: “*there is still room for improvement in sample efficiency*” [6].

B. Keterbatasan Eksplorasi dan Pendekatan CAE

Inefisiensi sampel pada algoritma DQN berakar dari mekanisme eksplorasi ϵ -greedy. Pada *environment* dengan *sparse reward* dan penalti fatal seperti *Minesweeper*, pemilihan *action* acak pada fase awal pelatihan cenderung memicu klik pada ranjau. Hal ini menyebabkan terminasi dini secara konstan, sehingga *agent* gagal mengakumulasi lintasan pengalaman (*trajectory*) yang informatif ke dalam *replay buffer* untuk memperbarui jaringan.

Sebagai solusi atas batasan eksplorasi acak, metode berbasis ketidakpastian (*uncertainty*) dikembangkan untuk mengarahkan *agent* pada *state* yang belum dikuasai. Li (2025) mengusulkan metode *Critic as Explorer* (CAE), sebuah strategi eksplorasi intrinsik yang memadukan pengukuran ketidakpastian langsung ke dalam arsitektur jaringan penilai (*value network*). Secara konseptual, CAE menggeser paradigma eksplorasi dengan memanfaatkan representasi fitur laten $\phi(s)$ yang diekstraksi dari lapisan *embedding* pada jaringan *critic* [7].

Berdasarkan distribusi representasi laten tersebut, CAE menghitung metrik ketidakpastian berbasis *Upper Confidence Bound* (UCB). Mekanisme ini menghasilkan *exploration bonus* β yang mencerminkan seberapa jarang fitur laten serupa diamati sebelumnya. Bonus ini kemudian diintegrasikan untuk memprioritaskan *action* yang mengarah pada *state* yang kurang terwakili secara fitur. Pendekatan ini memungkinkan *agent* untuk melakukan eksplorasi terarah sekaligus memiliki sifat adaptif (*self-annealing*), di mana bonus eksplorasi menurun secara natural seiring bertambahnya penguasaan *state* tanpa memerlukan penjadwalan manual. Walaupun CAE terbukti kompetitif pada *benchmark* kontrol kontinu, efektivitas dan kemampuan generalisasinya pada *environment* berbasis logika diskret dengan terminasi dini seperti *Minesweeper* belum dieksplorasi secara empiris, yang selanjutnya menjadi fokus utama dalam penelitian ini.

III. PERANCANGAN DAN IMPLEMENTASI SISTEM

A. Prosedur Pelatihan

Secara garis besar, penelitian mengikuti alur eksperimen yang terdiri dari lima tahapan utama: perancangan *environment*, implementasi *agent*, pelatihan dengan evaluasi berkala, pengujian akhir, dan analisis statistik. Tahap pertama adalah perancangan *environment Minesweeper* dengan papan 6×6 dan 4 ranjau, yang mengimplementasikan antarmuka *Gym-like* serta representasi *state* berbasis *one-hot encoding*. Tahap kedua berupa implementasi dua *agent* dengan arsitektur CNN yang identik, yaitu DQN dengan eksplorasi ϵ -greedy sebagai *baseline* dan CAE-DQN dengan bonus ketidakpastian UCB sebagai metode yang diusulkan. Selanjutnya, kedua *agent* dilatih selama 80,000 episode dengan *seed* tetap. Setiap 1,000 episode dilakukan evaluasi singkat menggunakan *policy greedy* untuk memantau *win rate* serta menyimpan model terbaik. Setelah pelatihan selesai, model terbaik dari masing-masing *agent* diuji pada 5 *seed* papan yang berbeda (masing-masing 500 episode) untuk mengukur kemampuan generalisasi (*policy greedy*). Terakhir, data yang terkumpul dianalisis menggunakan serangkaian uji statistik untuk menjawab rumusan masalah penelitian secara kuantitatif.

B. Desain Environment

Environment Minesweeper diimplementasikan dalam kelas *MinesweeperEnv* pada file *minesweeper_env.py*. Kelas ini dirancang mengikuti antarmuka standar *Gym-like* yang umum digunakan dalam penelitian RL, sehingga memudahkan integrasi dengan algoritma *agent* yang dikembangkan.

C. Konfigurasi Papan dan Ranjau

Environment menggunakan papan berukuran tetap 6×6 dengan jumlah ranjau sebanyak 4 buah, merujuk pada konfigurasi *Minesweeper* Wang & Lei (2025). Konfigurasi ini dipilih karena cukup kompleks untuk mengekspos kelemahan strategi eksplorasi acak, namun tetap memungkinkan pelatihan dilakukan dalam batasan komputasi yang wajar. Penempatan ranjau dilakukan secara acak menggunakan *pseudo-random number generator* yang dapat di-*seed* untuk menjamin reproduktibilitas eksperimen.

D. Representasi State: One-Hot Encoding

State yang diamati oleh *agent* direpresentasikan sebagai tensor tiga dimensi berukuran $6 \times 6 \times 10$. Secara intuitif, representasi ini dapat dibayangkan sebagai tumpukan 10 lembar peta transparan, masing-masing berukuran 6×6 petak. Setiap petak pada papan direpresentasikan oleh sebuah vektor sepanjang 10 elemen yang hanya berisi nilai 0 dan 1, mengikuti skema *one-hot encoding*. Tepat satu elemen bernilai 1 (aktif), sementara yang lain bernilai 0, seperti yang diilustrasikan pada Gambar 1. Posisi elemen yang aktif inilah yang menunjukkan kondisi petak tersebut. Sepuluh kanal (lembar transparan) tersebut memiliki peran sebagai berikut:

- **Kanal 0 sampai 8:** Merepresentasikan petak yang telah terbuka dengan nilai angka 0 hingga 8. Sebagai contoh, jika sebuah petak terbuka dan bernilai 3, maka hanya

kanal ke-3 yang bernilai 1, sedangkan kanal lainnya (0, 1, 2, 4, 5, 6, 7, 8, 9) bernilai 0.

- **Kanal 9:** Merepresentasikan petak yang **belum terbuka** (*unknown*). Jika petak masih tertutup, maka kanal ke-9 bernilai 1, sementara kanal lainnya bernilai 0.

Perlu diperhatikan bahwa tidak terdapat kanal khusus untuk ranjau. *Agent* tidak pernah mengamati ranjau secara langsung selama episode berlangsung. Jika *agent* memilih petak yang memiliki ranjau, episode akan langsung berakhir dan *agent* hanya menerima penalti, tanpa memperoleh representasi eksplisit mengenai lokasi ranjau tersebut.

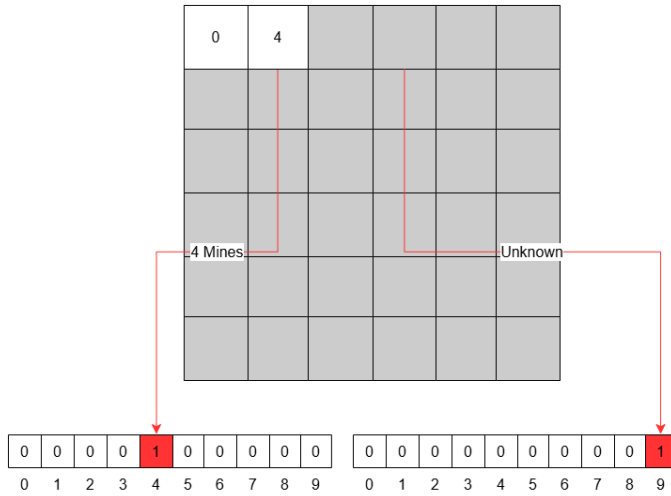


Fig. 1. Contoh representasi *One-Hot Encoding* dari *Minesweeper* 6×6

Dengan demikian, *agent* dipaksa untuk melakukan inferensi berdasarkan pola angka yang terungkap secara bertahap, serupa dengan cara manusia memainkan *Minesweeper*. Representasi *one-hot* ini dipilih karena mampu mempertahankan informasi spasial dan numerik secara eksplisit, serta dapat langsung diproses oleh arsitektur CNN yang digunakan sebagai model pembelajaran.

E. Struktur Reward

Fungsi *reward* dirancang untuk memberikan sinyal pembelajaran yang informatif meskipun dalam kondisi *sparse reward* alami pada permainan *Minesweeper*. Nilai *reward* untuk setiap instansi didefinisikan pada Tabel I

TABLE I
STRUKTUR *Reward Environment Minesweeper*

Kejadian	Nilai <i>Reward</i>	Deskripsi
Win	+36	<i>Agent</i> berhasil membuka semua petak aman.
Lose	-36	<i>Agent</i> memilih petak berisi ranjau.
Progress	+1.0	Membuka setidaknya satu petak aman baru.

F. Kondisi Terminasi dan Observabilitas Parsial

Sebuah episode dinyatakan berakhir (*done*) apabila salah satu dari dua kondisi berikut terpenuhi:

- 1) **Kalah:** *Agent* memilih petak yang berisi ranjau.
- 2) **Menang:** Seluruh petak yang tidak mengandung ranjau telah berhasil dibuka, yang secara ekuivalen terjadi ketika jumlah petak yang belum terbuka sama dengan jumlah ranjau (4).

Selain kondisi terminasi tersebut, *environment* ini juga merefleksikan karakteristik POMDP melalui observabilitas parsial. *Agent* tidak memiliki akses langsung terhadap papan solusi penuh (*self.board*), melainkan hanya terhadap representasi *state* dalam bentuk citra *one-hot* (*self.state_im*) pada waktu tertentu. Informasi mengenai lokasi ranjau serta nilai pada petak yang belum terbuka tetap tersembunyi, sehingga *agent* harus melakukan inferensi berdasarkan pola angka yang terungkap secara bertahap selama interaksi dengan *environment*.

G. Mekanisme Tambahan

Dua mekanisme tambahan diimplementasikan untuk meningkatkan stabilitas pelatihan sekaligus mencerminkan perilaku permainan standar:

- **Keamanan Klik Pertama:** Untuk menghindari terminasi instan, *environment* secara otomatis memindahkan ranjau jika *agent* memilih petak ranjau pada klik pertama, sehingga *agent* tetap dapat memulai proses penalaran.
- **Pembukaan Rekursif Nol:** Jika *agent* membuka petak bernilai 0, seluruh petak tetangganya akan terbuka secara rekursif hingga mencapai batas petak bernilai bukan 0.

Seluruh komponen *environment* dirancang deterministik untuk *seed* yang tetap, sehingga eksperimen dapat direproduksi secara konsisten dan memberikan fondasi yang seragam bagi pelatihan kedua *agent*.

TABLE II
RINGKASAN KONFIGURASI *Environment*

Parameter	Nilai
Ukuran Papan	6×6 petak
Jumlah Ranjau	4
Dimensi <i>State</i>	$6 \times 6 \times 10$ kanal
<i>Action Space</i>	36 diskret
<i>Seed Environment</i> (Pelatihan)	2004
<i>Seed Environment</i> (Evaluasi)	1, 2, 3, 4, 5

H. Desain Algoritma

Kedua *agent* dibangun menggunakan arsitektur CNN dan *hyperparameter* yang identik, sehingga strategi eksplorasi menjadi satu-satunya variabel pembeda. Dengan demikian, perbedaan kinerja dapat diatribusikan langsung pada mekanisme eksplorasi yang digunakan.

I. DQN

Agent DQN diimplementasikan sebagai *baseline* dalam penelitian ini, mengacu pada arsitektur klasik yang diperkenalkan oleh Mnih et al. (2015). Implementasi terdiri atas dua *file* utama, yaitu *dqn_model.py* untuk mendefinisikan arsitektur jaringan dan *dqn_agent.py* untuk menangani

pemilihan *action*, penyimpanan pengalaman, serta pembaruan parameter.

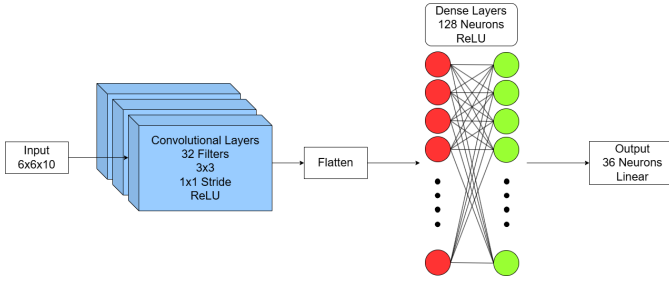


Fig. 2. Arsitektur CNN dari DQN pada Eksperimen

Model DQN dibangun menggunakan *TensorFlow/Keras* dengan arsitektur CNN yang terdiri atas tiga lapisan konvolusional dan dua lapisan *fully connected*, seperti ditunjukkan pada Gambar 2. Setiap lapisan konvolusional memiliki 32 *filter* berukuran 3×3 , *stride* 1×1 , *padding* *same*, dan aktivasi ReLU. Setelah *flatten*, representasi diproses oleh dua lapisan *fully connected* dengan 128 neuron dan aktivasi ReLU. Lapisan keluaran terdiri atas 36 neuron—sesuai jumlah petak pada papan 6×6 —dengan aktivasi linear untuk menghasilkan estimasi $Q(s, a)$. Arsitektur ini dipilih untuk menjaga efisiensi komputasi tanpa mengorbankan kapasitas representasi.

Selama pelatihan, *agent* memilih *action* menggunakan strategi ϵ -greedy. Dengan probabilitas ϵ , *agent* memilih *action* secara acak dari himpunan petak yang masih belum terbuka, sedangkan dengan probabilitas $1 - \epsilon$, *agent* memilih *action* dengan nilai Q tertinggi. Nilai ϵ diinisialisasi sebesar 0.95 dan meluruh secara eksponensial dengan faktor 0.99995 hingga mencapai batas minimum 0.02.

Untuk mencegah pemilihan *action* pada petak yang telah terbuka, diterapkan mekanisme *masking* dengan menetapkan nilai Q menjadi $-\infty$ pada indeks petak tersebut. Kondisi ini diidentifikasi melalui kanal ke-9 pada representasi *state*, di mana nilai 0 menunjukkan bahwa petak tidak lagi berstatus *unknown*. Dengan demikian, petak yang telah terbuka tidak akan pernah dipilih oleh operasi $\arg \max$. DQN selanjutnya menerapkan dua mekanisme utama:

1) *Experience Replay*

Transisi $(s_t, a_t, r_{t+1}, s_{t+1}, done)$ disimpan dalam *replay buffer* berkapasitas 300,000 sampel. Selama pelatihan, *batch* berukuran 128 diambil secara acak untuk memutus korelasi temporal dan memungkinkan penggunaan ulang pengalaman. Pelatihan dimulai setelah *buffer* terisi minimal 10,000 transisi.

2) *Target Network*

Jaringan terpisah dengan arsitektur identik digunakan sebagai *target network* untuk menghitung nilai target y_t . Parameter θ^- diperbarui dengan menyalin parameter jaringan utama θ setiap 1,000 langkah pelatihan (1 langkah pelatihan dilakukan setiap 4 langkah interaksi dengan *environment*). Mekanisme ini menjaga stabilitas target dan mengurangi risiko osilasi.

Pembaruan parameter jaringan utama dilakukan dengan meminimalkan MSE antara prediksi $Q(s_t, a_t; \theta)$ dan target y_t menggunakan *optimizer* Adam [9] dengan LR 0.001 dan parameter $\epsilon = 10^{-4}$. Pembaruan dilakukan setiap 4 langkah interaksi untuk menyeimbangkan akumulasi pengalaman dan frekuensi pelatihan. Pada setiap langkah pelatihan, nilai ϵ diperbarui dan penghitung pembaruan *target network* ditingkatkan, sehingga proses pembelajaran berlangsung secara berkala meskipun interaksi dengan *environment* terjadi secara kontinu.

J. CAE-DQN

Agent CAE-DQN mengintegrasikan mekanisme eksplorasi CAE ke dalam kerangka DQN yang telah dijelaskan sebelumnya. Implementasi terdiri atas dua *file* utama, yaitu *caedqn_model.py* yang mendefinisikan arsitektur jaringan dengan dua keluaran, serta *caedqn_agent.py* yang menangani logika pemilihan *action*, pembaruan *Gram matrix*, penyimpanan pengalaman, dan pembaruan parameter. Arsitektur CNN yang digunakan identik dengan *agent* DQN, terdiri atas tiga lapisan konvolusional dengan 32 *filter* berukuran 3×3 dan dua lapisan *fully connected* dengan 128 neuron. Perbedaan utama terletak pada pemanfaatan lapisan kedua terakhir sebagai lapisan *embedding* yang menghasilkan vektor fitur laten $\phi(s) \in \mathbb{R}^{128}$, yang merepresentasikan karakteristik *state*, seperti yang diilustrasikan oleh Gambar 3. Dengan demikian, model memiliki dua keluaran: nilai $Q(s, a)$ sepanjang 36 neuron dan vektor $\phi(s)$ yang digunakan khusus untuk menghitung bonus eksplorasi.

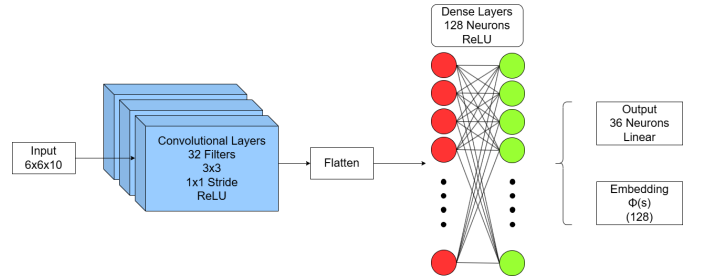


Fig. 3. Arsitektur CNN dari CAE-DQN pada Eksperimen

Selama pelatihan, *agent* CAE-DQN tidak lagi menggunakan strategi ϵ -greedy. Pada setiap langkah, *agent* menghitung bonus ketidakpastian $b(s)$ berdasarkan formulasi UCB pada ruang laten $\phi(s)$, dengan $\beta = 0.1$ sebagai koefisien eksplorasi dan $\mathbf{A}$ sebagai *Gram matrix* yang mengakumulasi informasi kovarians dari fitur laten *state* yang telah dikunjungi. Matriks $\mathbf{A}$ diinisialisasi sebagai $\mathbf{A} = \lambda \mathbf{I}$ dengan $\lambda = 1.0$ sebagai regularisasi *ridge*, dan diperbarui secara inkremental bersama inversnya $\mathbf{A}^{-1}$ menggunakan formula *Sherman–Morrison* untuk menjaga efisiensi komputasi. Untuk stabilitas numerik, bonus mentah dinormalisasi secara *running* menggunakan algoritma *Welford*, yang memperkirakan rerata dan variansi secara kumulatif.

Bonus yang telah dinormalisasi kemudian ditambahkan ke nilai $Q(s, a)$ untuk membentuk nilai teraugmentasi, yang

digunakan pada fase pemilihan *action*. Selain itu, bonus yang sama juga ditambahkan ke reward ekstrinsik r_{t+1} sebelum transisi disimpan ke dalam *replay buffer*, sehingga target pembelajaran y_t mencerminkan *augmented reward*. Modifikasi ini mendorong *agent* untuk tidak hanya memaksimalkan *reward* ekstrinsik, tetapi juga secara implisit mengurangi ketidaktahuan terhadap *state* yang belum banyak dieksplorasi.

K. Pelatihan

Pelatihan dilakukan selama 80,000 episode. Untuk menjamin reproduktibilitas pengukuran performa, *seed global environment* diinisialisasi ke 2004, namun penempatan ranjau (*mine placement*) diacak ulang pada setiap awal episode. Hal ini memastikan *agent* terpapar pada berbagai konfigurasi papan selama pelatihan. Ringkasan konfigurasi pelatihan serta skema pencatatan data disajikan pada Tabel III.

Selama evaluasi berkala, *agent* menggunakan *policy greedy* murni ($\epsilon = 0$, *no bonus CAE*) untuk mengukur kinerja eksploitasi secara objektif.

TABLE III
KONFIGURASI PELATIHAN DAN PENCATATAN DATA

Aspek	Keterangan
Total Episode	80,000
Seed Environment	2004
Seed Model	123
File Logging Per Episode Metrik	csv/training_[algoritma].csv episode, total_reward, progress, win, n_clicks, ϵ/β , learn_rate
File Evaluasi Berkala Frekuensi Evaluasi Metrik Evaluasi	csv/training_eval_[algoritma].csv Setiap 1,000 episode median progress, win rate, median reward, waktu
Penyimpanan Model Terbaik	[algoritma]_best.keras (berdasarkan win rate tertinggi)
Smoothing Kurva	SMA, window 500 episode

L. Evaluasi

Evaluasi akhir dilakukan terhadap model terbaik yang disimpan selama pelatihan, yaitu model dengan *win rate* tertinggi pada evaluasi berkala. Pengujian menggunakan 5 *seed* papan yang berbeda (1, 2, 3, 4, 5) untuk mengukur kemampuan generalisasi *agent* pada konfigurasi yang belum pernah dilihat sebelumnya. Ringkasan konfigurasi evaluasi disajikan pada Tabel IV.

Selama evaluasi, *agent* tidak melakukan eksplorasi maupun pembelajaran. Seluruh parameter jaringan dibekukan, dan *action* dipilih berdasarkan nilai $Q(s, a)$ tertinggi setelah *masking* petak yang telah terbuka. Hasil evaluasi ini menjadi dasar perhitungan metrik performa akhir seperti *win rate*, rata-rata *total reward*, dan rata-rata jumlah klik.

M. Metode Analisis

Data yang diperoleh dari pelatihan dan evaluasi dianalisis menggunakan serangkaian uji statistik dan metrik kuantitatif.

TABLE IV
KONFIGURASI EVALUASI AKHIR

Aspek	Keterangan
Model yang Digunakan	[algoritma]_best.keras (hasil pelatihan)
Seed Papan Evaluasi	1, 2, 3, 4, 5
Episode per Seed	500
Total Episode Evaluasi	2,500 per agent
Policy Pemilihan Action	Greedy murni ($\epsilon = 0$, tanpa bonus CAE)
File Hasil Evaluasi Metrik	csv/eval_[algoritma]_4mines.csv algorithm, board_size, n_mines, eval_seed, episode, win, progress, total_reward, n_clicks

Rangkuman metode analisis beserta tujuannya disajikan pada Tabel V.

Seluruh pengujian statistik dilakukan pada tingkat signifikansi $\alpha = 0.05$. Uji normalitas *Shapiro-Wilk* diterapkan terlebih dahulu untuk memastikan validitas penggunaan uji parametrik. Metode di atas dipilih agar setiap klaim yang diajukan dalam Bab IV, baik mengenai keunggulan performa, kecepatan belajar, stabilitas, maupun kualitas eksplorasi, didukung oleh bukti kuantitatif yang signifikan dan dapat dipertanggungjawabkan secara inferensial.

TABLE V
METODE ANALISIS KUANTITATIF YANG DIGUNAKAN

Kategori	Metode / Metrik	Tujuan
Performa Akhir	Welch's <i>t</i> -test	Menguji signifikansi perbedaan rata-rata <i>win rate</i> antara DQN dan CAE-DQN
	Cohen's <i>d</i>	Mengukur besaran efek (<i>effect size</i>) perbedaan performa
Interval Kepercayaan	Interval Parametrik	Mengestimasi rentang selisih <i>win rate</i> dengan asumsi normalitas
	Bootstrap (10k sampel)	Validasi interval kepercayaan tanpa asumsi distribusi
Efisiensi Sampel	AUC (40k episode)	Mengukur total akumulasi performa pada fase awal pelatihan
	Episode capai 50% <i>win rate</i>	Indikator kecepatan pembelajaran relatif
Tren Pelatihan	Mann-Kendall & Sen's	Mendeteksi keberadaan dan arah tren monoton pada kurva <i>win rate</i>
Stabilitas Akhir	Standar deviasi & regresi linier	Mengukur fluktuasi performa dan signifikansi penurunan pada fase lanjut
Kualitas Eksplorasi	Progress per click	Proporsi klik yang menghasilkan kemajuan
Adaptivitas Eksplorasi	Analisis bonus CAE	Memvalidasi sifat <i>self-annealing</i> bonus tanpa penjadwalan manual

IV. HASIL DAN ANALISIS

A. Data dan Hasil

Bagian ini menyajikan hasil dari seluruh eksperimen yang telah dilakukan. Evaluasi difokuskan pada tiga metrik utama: *total reward* per episode, jumlah klik per episode, dan *win rate*. Analisis performa dilakukan dengan membandingkan kurva pelatihan dari DQN standar dan CAE-DQN selama proses *training*, serta ditunjang oleh rangkuman metrik evaluasi akhir untuk mengukur efisiensi sampel dan kestabilan *agent*.

B. Kurva Pelatihan

Kurva pelatihan menyajikan perkembangan performa *agent* DQN dan CAE-DQN selama 80,000 episode eksperimen. Visualisasi ini mencakup metrik utama: *total reward*, *win rate*, dan *number of clicks* per episode. Seluruh data pada kurva ini diproses menggunakan metode *Simple Moving Average* (SMA) untuk mengurangi variansi statistik yang tinggi akibat penempatan ranjau secara acak, sehingga tren pembelajaran dan stabilitas konvergensi dari kedua model dapat dianalisis dengan lebih jelas.

Grafik 4 menunjukkan perbandingan metrik *total reward* antara model DQN standar dan CAE-DQN selama 80,000 episode, di mana CAE-DQN menunjukkan efisiensi sampel yang jauh lebih tinggi pada fase awal dengan mencapai puncak *reward* sekitar 25 sebelum episode 10,000, jauh melampaui DQN reguler yang baru mencapai nilai positif menjelang akhir pelatihan. Namun, meskipun mampu mencapai performa tinggi dengan sangat cepat, CAE-DQN mengalami fenomena *policy collapse* atau penurunan performa drastis setelah 50,000 episode, di mana nilai *reward* merosot kembali ke level dasar (≈ -30) tanpa menunjukkan tanda-tanda pemulihan.

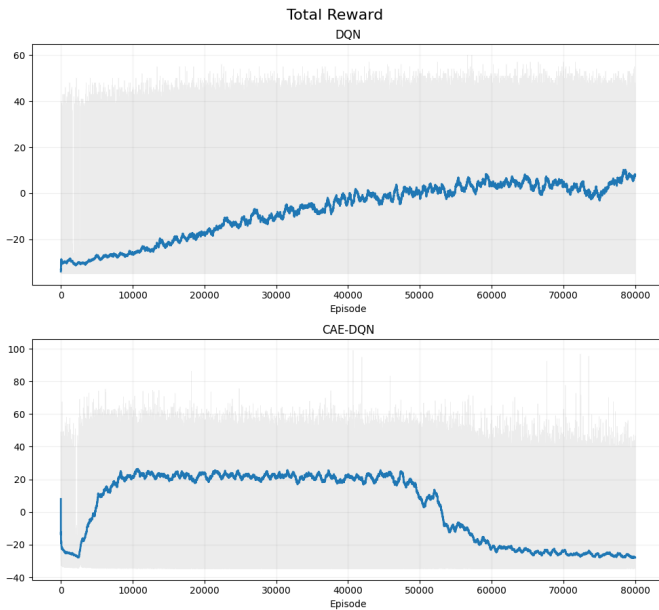


Fig. 4. *Total reward* dari pelatihan DQN dan CAE-DQN selama 80,000 episode

Grafik 5 menunjukkan hasil eksperimen jumlah klik selama 80,000 episode dengan trajektori perilaku yang berbeda bagi kedua *agent*. Model DQN standar menunjukkan tren kenaikan yang konsisten dan bertahap, dimulai dari sekitar 4 klik dan terus meningkat hingga mencapai rata-rata puncak hampir 7 klik pada akhir durasi pelatihan. Sebaliknya, CAE-DQN menunjukkan peningkatan jumlah klik yang pesat dalam 10,000 episode pertama, diikuti oleh fase stabil yang berkepanjangan di mana nilai rata-rata berfluktuasi di sekitar 7 klik. Namun, mulai sekitar episode 55,000, CAE-DQN menunjukkan penurunan rata-rata jumlah klik yang cukup terlihat, hingga akhirnya menetap di kisaran 5 klik pada episode 80,000, serta menampilkan variansi puncak yang lebih tinggi pada performa episode individual dibandingkan dengan DQN standar.

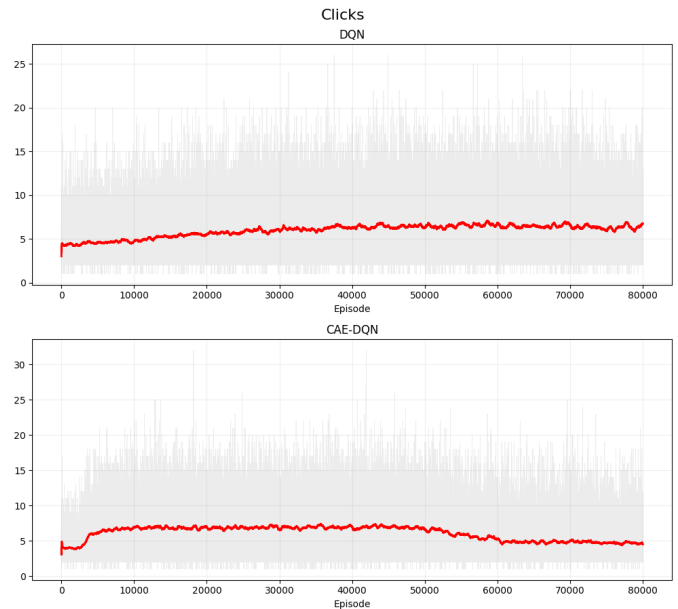


Fig. 5. *Number of clicks* dari pelatihan DQN dan CAE-DQN selama 80,000 episode

C. Kurva Evaluasi Pelatihan

Grafik 6 menunjukkan perbandingan *win rate* terhadap waktu pelatihan dari model DQN dan CAE-DQN. Grafik menunjukkan perbedaan performa yang signifikan antara DQN dan CAE-DQN pada permainan *Minesweeper* 6×6 . *Agent* CAE-DQN menunjukkan efisiensi pembelajaran yang sangat tinggi dengan lonjakan *win rate* yang tajam di awal pelatihan, mencapai nilai puncak sebesar 0.72 hanya dalam waktu 28.8 menit. Namun, setelah mencapai fase stabil (*plateau*), performa CAE-DQN mengalami penurunan drastis setelah menit ke-80 dan berakhir dengan *win rate* dibawah 0.10. Di sisi lain, *agent* DQN menunjukkan proses pembelajaran yang jauh lebih lambat namun stabil secara konsisten. DQN memerlukan waktu sekitar 204.4 menit untuk mencapai performa puncaknya dengan *win rate* sebesar 0.55. Meskipun nilai puncak DQN lebih rendah dibandingkan CAE-DQN, DQN tidak menunjukkan degradasi performa yang ekstrem hingga akhir durasi pelatihan yang diamati.



Fig. 6. Kurva *win rate* - *time* dari pelatihan DQN dan CAE-DQN selama 80,000 episode

D. Bonus Eksplorasi CAE

Grafik 7 menampilkan rata-rata bonus eksplorasi CAE per episode yang telah dihaluskan dengan SMA *window* 500 episode. Bonus dimulai dari nilai sekitar 10, menurun tajam ke kisaran 2 dalam 1,000 episode pertama, kemudian melanjutkan penurunan gradual hingga mencapai sekitar 0,5 pada episode ke-20,000. Sebuah lonjakan kecil teramati di sekitar episode ke-3,000 sebelum bonus kembali menurun secara konsisten.

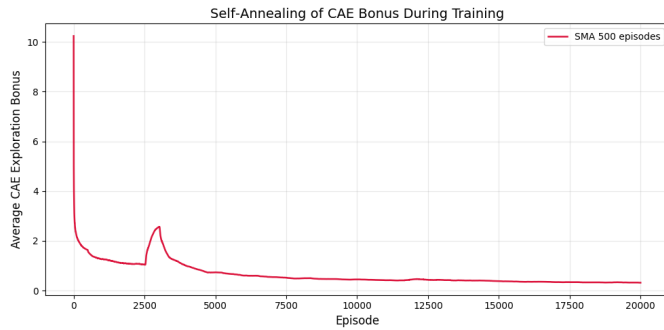


Fig. 7. *Self-annealing* bonus eksplorasi CAE selama 20,000 episode pelatihan.

E. Metrik Evaluasi

Tabel VI menyajikan rangkuman metrik evaluasi untuk model DQN dan CAE-DQN yang diuji dalam 500 episode untuk setiap *environment seed*. Secara keseluruhan, CAE-DQN menunjukkan performa yang lebih unggul dibandingkan dengan DQN standar pada ketiga metrik utama. CAE-DQN mencapai rata-rata *win rate* sebesar 72.6%, yang secara signifikan lebih tinggi dibandingkan DQN yang hanya mencapai 58.0%. Peningkatan ini juga tercermin pada *average total reward*, di mana CAE-DQN memperoleh nilai 22.214, hampir dua kali lipat dari nilai DQN yang hanya mencapai 11.370. Selain itu, CAE-DQN mencatat rata-rata jumlah klik yang sedikit lebih tinggi, yakni 6.942 klik dibandingkan dengan 6.610 klik pada DQN. Data per *seed* juga menunjukkan konsistensi performa pada kedua model, dengan variasi nilai yang relatif kecil di antara kelima *seed* yang diuji.

TABLE VI
RANGKUMAN METRIK EVALUASI MODEL DQN DAN CAE-DQN
SEBANYAK 500 EPISODE PER *seed*

DQN			
<i>Seed</i>	<i>Win Rate</i>	<i>Total Reward</i>	<i>Clicks</i>
1	59.0%	12.318	6.838
2	59.0%	12.058	6.578
3	56.6%	10.272	6.520
4	58.2%	11.530	6.626
5	57.2%	10.670	6.486
Overall	58.0%	11.370	6.610
CAE-DQN			
<i>Seed</i>	<i>Win Rate</i>	<i>Total Reward</i>	<i>Clicks</i>
1	72.6%	22.236	6.964
2	72.2%	21.880	6.896
3	72.4%	21.962	6.834
4	74.6%	23.840	7.128
5	71.2%	21.152	6.888
Overall	72.6%	22.214	6.942

F. Analisis

Bagian ini menyajikan analisis mendalam terhadap data yang telah dipaparkan pada bagian sebelumnya, dengan tujuan untuk menjawab pertanyaan penelitian yang diajukan serta mengungkap makna di balik pola-pola kuantitatif yang teramati. Analisis disusun secara bertahap, dimulai dari evaluasi performa akhir untuk mengukur signifikansi statistik keunggulan CAE-DQN atas DQN standar, dilanjutkan dengan pengujian efisiensi sampel melalui metrik AUC dan ambang batas *win rate* 50%. Selanjutnya, perilaku *self-annealing* bonus eksplorasi CAE dianalisis untuk menilai apakah mekanisme intrinsik tersebut mampu menggantikan penjadwalan eksplorasi manual. Kualitas eksplorasi diukur secara kuantitatif melalui rasio *progress per click* pada fase awal pelatihan untuk menentukan apakah *action* yang dihasilkan CAE lebih informatif dibandingkan pendekatan acak ϵ -greedy. Analisis kemudian ditutup dengan pemeriksaan terhadap stabilitas pelatihan jangka panjang, termasuk identifikasi penyebab penurunan performa CAE-DQN pada episode lanjut yang diduga kuat sebagai fenomena *policy collapse*. Setiap sub-analisis dilengkapi dengan uji statistik yang sesuai, seperti *Welch's t-test*, *Mann-Kendall*, regresi linier, dan perhitungan ukuran efek, sehingga kesimpulan yang dihasilkan tidak hanya bersifat deskriptif tetapi juga memiliki validitas inferensial yang dapat dipertanggungjawabkan.

G. Perbandingan Performa Akhir

Untuk menguji signifikansi perbedaan performa akhir antara DQN dan CAE-DQN, dilakukan uji statistik terhadap rata-rata *win rate* dari lima *seed* evaluasi yang masing-masing diuji sebanyak 500 episode.

Pertama, dilakukan uji normalitas *Shapiro-Wilk* pada data kedua kelompok. Uji ini bertujuan untuk menentukan apakah distribusi *win rate* per *seed* mengikuti distribusi normal, yang menjadi salah satu asumsi dasar bagi uji-t parametrik. Hasilnya menunjukkan bahwa distribusi *win rate* DQN ($W = 0.888$,

$p = 0.346$) dan CAE-DQN ($W = 0.901$, $p = 0.417$) tidak menyimpang secara signifikan dari distribusi normal ($p > 0.05$). Dengan demikian, uji-t independen (*independent samples t-test*) dengan koreksi *Welch* dapat digunakan secara valid untuk membandingkan kedua rata-rata.

Uji-t independen digunakan untuk menguji hipotesis nol bahwa tidak terdapat perbedaan rata-rata *win rate* yang signifikan antara kedua kelompok. Hasil uji-t menunjukkan bahwa CAE-DQN ($M = 72.6\%$, $SD = 1.24\%$) secara signifikan mengungguli DQN ($M = 58.0\%$, $SD = 1.08\%$), $t(8) = 19.87$, $p < 0.001$. Dengan demikian, hipotesis nol (H_0) ditolak dan hipotesis alternatif (H_1) diterima, CAE-DQN secara signifikan meningkatkan *win rate* dibandingkan DQN standar.

Untuk mengukur besaran perbedaan secara praktis, dihitung ukuran efek *Cohen's d*, yang mengkuantifikasi seberapa besar perbedaan rata-rata kedua kelompok relatif terhadap variabilitas gabungan data. Nilai *Cohen's d* yang diperoleh sebesar 12.57, yang menurut kriteria konvensional ($d = 0.2$ kecil, $d = 0.5$ sedang, $d = 0.8$ besar) termasuk dalam kategori **sangat besar**. Hal ini mengindikasikan bahwa keunggulan CAE-DQN tidak hanya signifikan secara statistik, tetapi juga sangat substansial secara praktis. (**Catatan:** Karena evaluasi dilakukan pada lima *seed* papan yang berbeda, variabilitas antar *seed* relatif kecil sehingga nilai *Cohen's d* yang diperoleh cenderung tinggi. Meskipun demikian, selisih absolut *win rate* sebesar 14.6% tetap menunjukkan peningkatan performa yang nyata.)

Selanjutnya, untuk mengestimasi rentang nilai selisih *win rate* yang mungkin terjadi pada populasi, dihitung interval kepercayaan 95% parametrik. Interval kepercayaan 95% untuk selisih rata-rata *win rate* adalah [12.90%, 16.30%], yang berarti kita dapat yakin 95% bahwa selisih rata-rata sebenarnya berada dalam rentang tersebut. Sebagai validasi tambahan, dilakukan analisis *bootstrap* dengan 10,000 kali pengambilan sampel ulang, yang menghasilkan interval kepercayaan serupa [13.40%, 15.92%]. Metode *bootstrap* tidak bergantung pada asumsi distribusi normal dan memberikan estimasi interval yang lebih *robust*.

Dengan demikian, berdasarkan seluruh rangkaian uji statistik, dapat disimpulkan bahwa penambahan mekanisme eksplorasi CAE ke dalam arsitektur DQN menghasilkan peningkatan performa akhir yang sangat signifikan dan praktis bermakna (*practically significant*) pada *environment Minesweeper* 6×6 dengan 4 ranjau.

H. Efisiensi Sampel dan Kecepatan Pembelajaran

Untuk mengukur keunggulan CAE-DQN dalam hal efisiensi sampel (*sample efficiency*) dan kecepatan pembelajaran, dilakukan tiga analisis kuantitatif terhadap data *win rate* yang dicatat setiap 1,000 episode selama pelatihan. Analisis ini mencakup perhitungan AUC, penentuan episode pertama kali mencapai ambang batas 50% *win rate*, serta uji tren *Mann-Kendall*.

Pertama, AUC dihitung dengan mengintegrasikan kurva *win rate* terhadap episode menggunakan metode trapesium pada

40,000 episode pertama. Dalam konteks eksperimen ini, AUC merepresentasikan total akumulasi performa yang berhasil dikumpulkan oleh *agent* selama fase awal pelatihan. Semakin tinggi nilai AUC, semakin cepat *agent* belajar dan semakin tinggi *win rate* yang mampu dipertahankan pada periode tersebut. Hasil perhitungan menunjukkan bahwa CAE-DQN mencapai AUC sebesar 24,135, sementara DQN standar hanya memperoleh AUC sebesar 7,835. Dengan kata lain, CAE-DQN mengakumulasi performa pembelajaran sekitar 3.08 kali lipat lebih besar dibandingkan DQN pada rentang episode yang sama. Perbedaan yang sangat mencolok ini menegaskan bahwa mekanisme eksplorasi berbasis UCB pada CAE secara drastis mempercepat perolehan *reward* dan kemenangan pada fase awal pelatihan.

Kedua, untuk mengukur kecepatan pembelajaran secara lebih intuitif, ditentukan episode pertama di mana masing-masing *agent* berhasil mencapai *win rate* ≥ 0.5 pada evaluasi berkala. CAE-DQN mencapai ambang batas ini pada episode ke-6,000, yang berarti hanya dalam 6,000 episode pertama ($\approx 7.5\%$ dari total pelatihan) *agent* telah mampu memenangkan lebih dari separuh permainan evaluasi. Sebaliknya, DQN baru berhasil menembus ambang batas 50% pada episode ke-65,000, atau setelah menghabiskan lebih dari 80% total episode pelatihan. Perbedaan kecepatan sebesar satu orde magnitudo ini (6,000 vs. 65,000 episode) merupakan bukti kuantitatif yang sangat kuat bahwa CAE memberikan peningkatan efisiensi sampel yang substansial pada *environment Minesweeper* dengan *reward* jarang dan penalti fatal.

Ketiga, untuk mengonfirmasi keberadaan tren peningkatan atau penurunan performa secara statistik sepanjang keseluruhan pelatihan (80,000 episode), dilakukan uji tren non-parametrik *Mann-Kendall*. Uji ini menguji hipotesis nol bahwa tidak terdapat tren monoton (naik atau turun) pada deret data, tanpa mensyaratkan asumsi distribusi tertentu, sehingga *robust* terhadap fluktuasi acak yang umum terjadi pada kurva pembelajaran RL. Hasil uji menunjukkan bahwa DQN memiliki tren yang **meningkat** secara signifikan ($p < 0.001$, *Sen's slope* = 0.0065). Hal ini mengindikasikan bahwa meskipun lambat, DQN secara konsisten menunjukkan perbaikan performa seiring bertambahnya pengalaman. Sebaliknya, CAE-DQN menunjukkan tren yang **menurun** secara signifikan ($p < 0.001$, *Sen's slope* = -0.0071). Tren negatif ini mengonfirmasi secara statistik bahwa performa pelatihan CAE-DQN mengalami degradasi setelah mencapai puncaknya.

Secara keseluruhan, ketiga metrik di atas secara konsisten menunjukkan bahwa CAE-DQN unggul secara dramatis dalam hal efisiensi sampel dan kecepatan pembelajaran awal. Adapun penurunan performa yang teramati pada fase akhir pelatihan CAE-DQN, yang tercermin dari tren *Mann-Kendall* negatif, tidak serta-merta mengindikasikan ketidakstabilan inheren metode CAE. Penurunan tersebut lebih tepat diatribusikan pada fenomena *policy collapse*. Meskipun performa pada papan evaluasi yang berbeda tetap tinggi (Tabel VI). Sementara itu, DQN yang belajar lebih lambat belum menunjukkan gejala *policy collapse* hingga akhir pelatihan, mencapai *plateau* yang stabil di sekitar *win rate* 55%.

I. Adaptivitas Eksplorasi: Self-Annealing Bonus CAE

Penurunan bonus yang teramati pada Grafik 7 dapat dijelaskan melalui mekanisme internal CAE. Bonus dihitung sebagai $\beta \sqrt{\phi^\top A^{-1} \phi}$, di mana ϕ adalah vektor fitur laten dari *state* saat ini, dan A adalah *Gram matrix* yang mengakumulasi informasi seluruh *state* yang pernah dikunjungi. Pada awal pelatihan, matriks A hanya berisi regularisasi λI , sehingga invers A^{-1} masih besar dan menghasilkan bonus yang tinggi. Seiring *agent* berinteraksi dengan *environment*, fitur ϕ dari *state* yang sering muncul terus ditambahkan ke dalam A melalui pembaruan *Sherman–Morrison*. Akumulasi ini menyebabkan A^{-1} mengecil secara gradual, yang pada gilirannya menekan nilai bonus.

Penurunan tajam dalam 1,000 episode pertama terjadi karena *agent* dengan cepat mengeksplorasi *state-state* dasar yang mendominasi fase awal permainan. Setelah *state-state* tersebut terwakili dengan baik dalam *Gram matrix*, bonus menurun ke level yang lebih rendah. Lonjakan kecil di sekitar episode ke-3,000 mengindikasikan bahwa *agent* mulai menghadapi konfigurasi papan yang lebih kompleks dan belum pernah ditemui sebelumnya. Karena fitur ϕ dari *state* baru tersebut belum terakumulasi dalam A , ketidakpastian meningkat sementara dan bonus ikut naik. Setelah *state* baru tersebut cukup sering dikunjungi, informasi kembali terakumulasi dan bonus melanjutkan tren penurunan.

Pada akhir 20,000 episode, bonus telah menyusut ke sekitar 0.5, menunjukkan bahwa *agent* telah membangun representasi yang cukup komprehensif terhadap *state space* yang relevan. Dengan demikian, mekanisme CAE secara intrinsik mampu menyesuaikan intensitas eksplorasi berdasarkan tingkat kebaruan *state* yang dihadapi, tanpa memerlukan penjadwalan eksternal.

J. Kualitas Eksplorasi: Progress per Click

Sebagai indikator kualitas eksplorasi, dihitung metrik *progress per click*, yaitu **proporsi klik yang menghasilkan kemajuan** (rasio antara jumlah klik yang menerima *reward* *progress* terhadap total klik), pada 10,000 episode pertama pelatihan. Metrik ini mengukur seberapa efisien setiap keputusan eksplorasi dalam menambah pengetahuan *agent* tentang papan. Hasil perhitungan menunjukkan bahwa CAE-DQN mencatat rata-rata *progress per click* sebesar 0.828, sementara DQN standar memperoleh 0.734. Secara relatif, CAE-DQN mengungkap sekitar 13% lebih banyak petak baru per klik dibandingkan DQN pada fase awal pelatihan.

Meskipun peningkatan sebesar 13% tampak moderat, perlu dipertimbangkan bahwa pada tahap awal permainan *Minesweeper*, sebagian besar klik secara alami akan membuka sel baru karena sebagian besar papan masih tertutup. Oleh karena itu, ruang untuk perbaikan absolut pada metrik ini memang terbatas. Namun, konsistensi CAE-DQN dalam menghasilkan *progress per click* yang lebih tinggi—terlihat dari median sebesar 0.857 dibandingkan 0.750 pada DQN—mengindikasikan bahwa eksplorasi CAE lebih terarah. *Agent* CAE-DQN cenderung mengklik petak dengan

ketidakpastian tinggi yang lebih mungkin membuka area baru di papan.

Dengan demikian, data *progress per click* mendukung pandangan bahwa mekanisme bonus UCB pada CAE tidak hanya mempercepat konvergensi secara keseluruhan, tetapi juga meningkatkan kualitas setiap langkah eksplorasi yang diambil, menjadikannya lebih informatif dibandingkan pendekatan ϵ -greedy.

K. Stabilitas Pelatihan dan Fenomena Penurunan

Untuk menganalisis secara kuantitatif fenomena penurunan performa CAE-DQN yang teramati pada kurva pelatihan setelah episode ke-50,000, dilakukan dua analisis terhadap data *win rate* pada rentang episode 50,000 hingga 80,000: perhitungan variansi sebagai indikator stabilitas, serta regresi linier untuk menguji signifikansi tren penurunan. Perlu dicatat bahwa analisis ini bertujuan untuk mengkarakterisasi dinamika pelatihan dan mendiagnosis penyebab penurunan, bukan untuk mengevaluasi model akhir yang digunakan. Model CAE-DQN yang dievaluasi pada Tabel VI disimpan pada titik performa puncak (sekitar episode ke-6,000–10,000), sehingga pelatihan lanjutan yang dianalisis di sini tidak memengaruhi kualitas *agent* yang diuji pada *seed* berbeda.

Pertama, variansi *win rate* pada 30,000 episode terakhir dihitung untuk menilai kestabilan masing-masing *agent* setelah fase konvergensi awal. Dalam konteks ini, standar deviasi yang lebih rendah mengindikasikan performa yang lebih stabil dan konsisten dari episode ke episode. Hasil perhitungan menunjukkan bahwa DQN memiliki standar deviasi sebesar 0.0297, sedangkan CAE-DQN memiliki standar deviasi sebesar 0.1524, sekitar lima kali lipat lebih besar. Perbedaan ini menegaskan bahwa DQN mempertahankan performa yang jauh lebih stabil pada fase akhir pelatihan, sementara CAE-DQN mengalami fluktuasi yang ekstrem seiring dengan penurunan performanya. Rata-rata *win rate* pada rentang yang sama juga mencerminkan hal ini: DQN mempertahankan rata-rata 46.9%, sedangkan CAE-DQN merosot hingga rata-rata 16.6%.

Kedua, untuk menguji apakah penurunan tersebut signifikan secara statistik, dilakukan analisis regresi linier sederhana dengan model $y = \beta_0 + \beta_1 \cdot \text{episode}$ pada data *win rate* CAE-DQN untuk rentang episode 50,000 hingga 80,000. Uji ini bertujuan untuk menentukan apakah terdapat tren linier yang signifikan (koefisien $\beta_1 \neq 0$) pada segmen penurunan. Hasil regresi menunjukkan bahwa koefisien kemiringan (*slope*) $\beta_1 = -0.000014$ bernilai negatif dan signifikan secara statistik ($p < 0.001$), dengan nilai $R^2 = 0.7027$. Temuan ini mengonfirmasi bahwa penurunan *win rate* CAE-DQN pada fase akhir pelatihan bukanlah fluktuasi acak, melainkan tren degradasi yang sistematis dan terprediksi.

Meskipun hasil di atas menunjukkan penurunan performa yang nyata selama pelatihan, penting untuk membedakan antara degradasi pada *environment* pelatihan dan kegagalan metode CAE secara fundamental. Jika penurunan ini disebabkan oleh ketidakstabilan inheren algoritma CAE, maka performa pada *environment* evaluasi yang berbeda seharusnya

juga ikut runtuh. Namun, berdasarkan Tabel VI, *agent* CAE-DQN yang dievaluasi pada lima *seed* papan berbeda tetap mampu mencapai rata-rata *win rate* sebesar 72.6%. Kesenjangan antara performa pelatihan yang merosot dan performa evaluasi yang tetap tinggi ini mengindikasikan secara kuat bahwa fenomena tersebut merupakan bentuk *policy collapse*, bukan ketidakstabilan sistemis pada metode CAE.

Penjelasan fenomena *collapse* tersebut dapat dikaitkan dengan dinamika *replay buffer*. Dengan kapasitas *buffer* sebesar 300,000 transisi dan rata-rata panjang lintasan sekitar 6–7 langkah per episode, *replay buffer* mencapai kapasitas jenuh tepat pada kisaran episode 45,000 hingga 50,000. Pada titik jenuh ini, mekanisme *First-In-First-Out* (FIFO) mulai melakukan penghapusan terhadap data terlama dalam *buffer*. Data terlama yang dikeluarkan adalah transisi dari fase awal pelatihan (0–10,000 episode), yaitu periode di mana bonus eksplorasi CAE masih sangat tinggi ($b(s) \approx 10$) dan *agent* menghasilkan trajektori yang sangat beragam dalam ruang representasi laten. Saat data lama yang kaya akan variansi ini tergusur sepenuhnya oleh transisi deterministik dari fase *self-annealing* (di mana bonus mendekati 0), distribusi data dalam *buffer* menjadi homogen dan hanya mencerminkan *policy* lokal yang bersifat *greedy*.

Tanpa akses terhadap keragaman pengalaman masa lalu, CNN rentan mengalami *catastrophic forgetting*. Parameter jaringan mulai melupakan fitur-fitur kritis dari area *state space* yang sudah lama tidak dikunjungi, menyebabkan estimasi *Q-value* menjadi *corrupted* dan mendorong *policy* ke dalam *local minimum* yang sub-optimal. Fenomena ini sekaligus menjelaskan mengapa *baseline* DQN tidak mengalami keruntuhan serupa. Strategi ϵ -*greedy* mempertahankan nilai $\epsilon_{\min} = 0,02$, sehingga terdapat injeksi data eksplorasi acak yang konstan ke dalam *buffer*. Injeksi ini bertindak sebagai *regularizer* alami yang mencegah homogenisasi total dari distribusi data dan melindungi stabilitas aproksimasi fungsi jaringan.

Oleh karena itu, meskipun *agent* terjebak dalam siklus kesalahan yang sama pada akhir pelatihan, model yang disimpan pada titik performa puncak sebelum terjadinya *collapse* tetap memiliki kemampuan generalisasi yang unggul, dibuktikan oleh *win rate* evaluasi sebesar 72.6%.

V. KESIMPULAN

Berdasarkan hasil eksperimen dan analisis, beberapa kesimpulan utama dapat dirumuskan. Pertama, CAE-DQN berhasil merealisasikan mekanisme eksplorasi berbasis ketidakpastian sesuai formulasi CAE. Pembaruan *Gram matrix* via *Sherman–Morrison* dan normalisasi *Welford* berjalan stabil, menunjukkan integrasi efektif ke dalam kerangka DQN.

Kedua, CAE-DQN secara signifikan mengungguli DQN standar, dengan rata-rata *win rate* 72.6% vs. 58.0% ($p < 0.001, d = 12.57$). Ambang *win rate* 50% dicapai dalam 6,000 episode, jauh lebih cepat dibandingkan 65,000 episode pada DQN, dengan AUC 24.135 vs. 7.835 yang mengonfirmasi keunggulan efisiensi pembelajaran.

Ketiga, bonus eksplorasi CAE menunjukkan *self-annealing* yang konsisten, menurun dari ≈ 10 ke ≈ 0.5 dalam 20,000

episode tanpa penjadwalan *decay* manual, menandakan transisi alami dari eksplorasi ke eksploitasi.

Keempat, eksplorasi CAE lebih terarah, tercermin dari peningkatan *progress per click* sebesar 13% pada fase awal. *Agent* cenderung memilih *state* dengan ketidakpastian tinggi, mempercepat akuisisi pengetahuan tentang *environment*.

Kelima, penurunan performa pada fase lanjut merupakan *policy collapse* akibat menyusutnya bonus CAE dan homogenisasi *replay buffer*. Namun, model yang disimpan pada titik puncak (sekitar episode 6.000–10.000) tetap menunjukkan generalisasi yang baik pada *seed* evaluasi berbeda.

Secara keseluruhan, CAE terbukti sebagai strategi eksplorasi yang efektif untuk *environment* dengan observasi parsial, *reward* jarang, dan penalti terminal seperti *Minesweeper*.

A. Saran

Berdasarkan keterbatasan yang teridentifikasi, saran untuk penelitian selanjutnya adalah:

- 1) **Konfigurasi Papan Lebih Kompleks:** Menguji skalabilitas CAE-DQN pada papan lebih besar seperti 9×9 dengan 10 ranjau atau 16×16 dengan 40 ranjau.
- 2) **Penerapan CAE+:** Untuk *environment* dengan *reward* semakin jarang, CAE+ dengan jaringan bantu *Inverse Dynamics* dapat dipertimbangkan guna mempercepat pembelajaran representasi laten.
- 3) **Aplikasi pada Domain Logika Lain:** Menerapkan kerangka CAE-DQN pada permainan seperti *Battleship*, *Mastermind*, atau *nonogram* untuk menguji generalitas temuan.
- 4) **Studi Ablasi Normalisasi Welford:** Mengevaluasi kontribusi normalisasi *Welford* terhadap stabilitas pelatihan secara kuantitatif.

REFERENCES

- [1] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski *et al.*, “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.
- [2] R. S. Sutton and A. G. Barto, *Reinforcement learning: An introduction*, 2nd ed. MIT press, 2018.
- [3] M. T. Spaan, “Partially observable Markov decision processes,” in *Reinforcement Learning: State-of-the-Art*. Springer, 2012, pp. 387–414.
- [4] L. P. Kaelbling, M. L. Littman, and A. R. Cassandra, “Planning and acting in partially observable stochastic domains,” *Artificial intelligence*, vol. 101, no. 1-2, pp. 99–134, 1998.
- [5] R. Kaye, “Minesweeper is NP-complete,” *The Mathematical Intelligencer*, vol. 22, no. 2, pp. 9–15, 2000.
- [6] W. Wang and C. Lei, “Training a minesweeper agent with deep q-network and supervised learning,” *Applied Sciences*, vol. 15, no. 3, p. 745, 2025.
- [7] Y. Li *et al.*, “CAE: Repurposing the critic as an explorer in deep reinforcement learning,” *arXiv preprint arXiv:2503.18980*, 2025, also presented at relevant conference.
- [8] L. Gardea, G. Koontz, and R. Silva, “Training a minesweeper solver,” *CS 229 Project Report, Stanford University*, 2015.
- [9] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*, 2015.